BumberoOS Server

System Guide

How the system works, its REST API, and a worked example from reporting an incident to resolving it.

What it is	Back-office REST API for a volunteer fire brigade: staff, incidents, equipment and logins.
Stack	Java 17, Spring Boot 4.1.1, Spring Data JPA, PostgreSQL (Neon); H2 for tests.
Base URL	http://localhost:8080/api (JSON in, JSON out)
Branch	majorRevisoin (dispatch, status sync and incident classification revision)

1. What the system does

BumberoOS tracks four things and keeps them consistent with each other:

- **Staff** - volunteers, with a stored availability (UNAVAILABLE, AVAILABLE, DISPATCHED).
- **Items** - engines, ambulances, SCBA sets, fire suits, hoses and so on, tracked by quantity owned and quantity currently dispatched.
- **Incidents** - each is an EMERGENCY or NON_EMERGENCY and is ONGOING or RESOLVED. Emergencies can carry a BFP alarm level with a history of escalations.
- **Accounts** - dispatcher/admin logins, optionally linked to a staff member.

The core idea is **status sync**. A dispatcher assigns staff and items to an incident; while they are assigned, staff are DISPATCHED and the item units count as dispatched. When the assignment is released, or the incident is resolved or deleted, they return to AVAILABLE automatically. Nobody has to update availability by hand.

> **The brigade records, it does not declare.** Alarm levels (first alarm up to General Alarm) are declared by the BFP. The system stores the level and every escalation with its timestamp so the history is available later.

2. How it works

Architecture

```
HTTP client (JSON)
      |
      v
controllers/    REST endpoints, request validation (@Valid), error mapping
      |
      +--> IncidentService     plain incident CRUD, PATCH, filters, alarm history
      +--> DispatchService     assign / release / resolve / reopen / delete (keeps availability in sync)
      +--> StaffService, InventoryService, UserAccountService, AuthenticationService
      |
      v
mappers/        DTO <-> entity conversion
repositories/   Spring Data JPA (plus row-locking queries)
      |
      v
PostgreSQL (Neon)   -- H2 in-memory for tests
```

- Entities never leave the API: controllers accept *Request* records and return *Response* records.
- IDs are generated by the server; any id sent on create is ignored.
- **PATCH** updates are partial: an omitted or null field is left unchanged.
- Text is normalised on save (names Title Case, phone numbers digits only, emails lower-case), so values read back can differ from what was sent.
- Every error comes back as `{"error": "message"}` with a specific, readable message.

Dispatch rules

Action	Staff member	Item
Assign	Must be employed and AVAILABLE. Becomes DISPATCHED.	Must be in GOOD condition with enough free units. Dispatched count goes up.
Release	Becomes AVAILABLE.	Units go back to the pool.
Resolve incident	Everyone on it is released and becomes AVAILABLE.	All units return. Optionally record a return condition (GOOD, DAMAGED, RETIRED).
Delete incident	If still ONGOING, released first, then deleted.	Same: units returned first.

- **Atomic create:** creating an incident with staff and items either fully succeeds or changes nothing. If one person is already dispatched, no incident is created and nobody else is touched.
- **One incident at a time:** a staff member can be active on only one incident. Item units can never exceed what is in stock.
- **Items are quantity-based:** 10 fire suits is one row. Set the quantity on an incident to the total you want there (4, then 6 sends two more). Track an engine as its own row with quantity 1.
- **History is kept:** releasing sets a release time instead of deleting the row, so you can see who and what was on scene and when.
- **Reopen** turns a resolved incident back to ONGOING but does not re-dispatch anyone.
- **Races are safe:** rows are locked while assigning (incident first, then staff, then items, in id order). If two dispatchers go for the same person or last engine at once, one succeeds and the other gets a 409.

Incident classification

- **Nature** (required): EMERGENCY or NON_EMERGENCY (standby, drills, water rationing).
- **Status**: always starts ONGOING. It can only change through the resolve and reopen calls, never by PATCH, so the release logic cannot be bypassed.
- **Alarm level** (emergencies only): FIRST_ALARM to FIFTH_ALARM, TASK_FORCE_ALPHA to INDIA, GENERAL_ALARM. Each change is appended to the incident's alarm history, and downgrades are allowed.
- Switching an incident to NON_EMERGENCY clears its alarm level. A resolved incident's nature and alarm level cannot be edited (reopen it first).
- Optional timestamps: *underControlAt*, *fireOutAt*; *resolvedAt* is set by resolve.

3. Data model

Table	Key columns
`staff`	id, name, role, contactNumber, address, employed, staffAvailability, dateJoined; child tables for trainings and certifications
`items`	id, name, description, category, serialNumber, itemCondition, quantity, quantityDispatched, lastMaintained
`incidents`	id, nature, status, alarmLevel, incidentType, location, reportedBy, dateTimeReported, underControlAt, fireOutAt, resolvedAt (severityLevel is deprecated and optional)
`incident_staff_assignments`	incident, staff, role (e.g. Driver), assignedAt, releasedAt (empty = active)
`incident_item_assignments`	incident, item, quantity, assignedAt, releasedAt, returnCondition
`incident_alarm_events`	incident, alarmLevel, raisedAt
`user_accounts`	id, email (unique), passwordHash (BCrypt), role, displayName, enabled, staff (optional), createdAt

Values the API accepts

Field	Allowed values
`StaffAvailability`	UNAVAILABLE, AVAILABLE, DISPATCHED
`IncidentNature`	EMERGENCY, NON_EMERGENCY
`IncidentStatus`	ONGOING, RESOLVED
`AlarmLevel`	FIRST_ALARM, SECOND_ALARM, THIRD_ALARM, FOURTH_ALARM, FIFTH_ALARM, TASK_FORCE_ALPHA, TASK_FORCE_BRAVO, TASK_FORCE_CHARLIE, TASK_FORCE_DELTA, TASK_FORCE_ECHO, TASK_FORCE_HOTEL, TASK_FORCE_INDIA, GENERAL_ALARM
`ItemCategory`	ENGINE, AMBULANCE, FIRST_AID, SCBA, FIRE_SUIT, HOSE, RESCUE_TOOL, OTHER
`ItemCondition`	GOOD, DAMAGED, RETIRED
`ItemAvailability (response only)`	AVAILABLE, PARTIALLY_DISPATCHED, DISPATCHED, UNAVAILABLE
`UserRole`	ADMIN, DISPATCHER

An item's **availability** is worked out, not stored: UNAVAILABLE if it is not GOOD or has no stock; AVAILABLE if nothing is out; DISPATCHED if every unit is out; otherwise PARTIALLY_DISPATCHED. *quantityAvailable* = quantity - quantityDispatched.

4. API reference

All requests and responses are JSON (`Content-Type: application/json`). Base path `/api`. An asterisk (*) marks a required field.

Errors

Status	When
400	Validation failure; alarm level on a non-emergency incident; item quantity below 1; item return for an item not on the incident; quantity would go below zero.
401	Sign-in failed (unknown email, wrong password or disabled account - same message for all).
404	Incident, staff, item or account not found.
409	State conflict: staff not AVAILABLE or not employed; item not GOOD or not enough units; incident not ONGOING; deleting staff or items that have assignment history; reducing quantity below what is dispatched; duplicate email on sign-up.

```
{ "error": "Ana is DISPATCHED" }
{ "error": "only 4 of Fire Suit available, 5 requested" }
{ "error": "incident 999 not found" }
```

Incidents - /api/incidents

Method	Path	Purpose
GET	`/`	List. Optional filters: ?status=ONGOING|RESOLVED & nature=EMERGENCY|NON_EMERGENCY.
GET	`/{id}`	One incident with its full assignment and alarm history.
POST	`/`	Create (201). Optionally assigns staff and items in the same call, atomically.
PATCH	`/{id}`	Edit nature, alarm level, text fields, timestamps. Cannot change status.
PUT	`/{id}/staff/{staffId}?role=Driver`	Assign a staff member (idempotent; a repeat only updates role).
DELETE	`/{id}/staff/{staffId}`	Release a staff member (no-op if not active).
PUT	`/{id}/items/{itemId}?quantity=N`	Set the total units of that item on this incident (N >= 1). More takes extra units; fewer returns some.
DELETE	`/{id}/items/{itemId}`	Release all units of that item.
POST	`/{id}/resolve`	Resolve: releases everything, returns staff and units. Optional body below.
POST	`/{id}/reopen`	Back to ONGOING. Does not re-dispatch anyone.
DELETE	`/{id}`	Delete (204). Releases anything still dispatched first.

Request bodies

```
POST /api/incidents
{
  "nature": "EMERGENCY",                 // *   EMERGENCY | NON_EMERGENCY
  "alarmLevel": "FIRST_ALARM",           //     emergencies only
  "incidentType": "Fire",                // *
  "location": "Zone 3",                  // *
  "reportedBy": "J. Cruz",               // *
  "dateTimeReported": "2026-10-14T09:30:00",   // optional, defaults to now
  "staffIds": [1, 2],                    //     optional
  "items": [ { "itemId": 1, "quantity": 1 }, { "itemId": 2, "quantity": 4 } ]   // optional
}

PATCH /api/incidents/{id}     (any of these; omitted = unchanged)
{ "nature", "alarmLevel", "incidentType", "location", "reportedBy",
  "dateTimeReported", "underControlAt", "fireOutAt" }

POST /api/incidents/{id}/resolve     (body optional)
{ "itemReturns": [ { "itemId": 1, "condition": "DAMAGED" } ] }
```

Response shape (IncidentResponse)

`id, nature, status, alarmLevel, incidentType, location, reportedBy, dateTimeReported, underControlAt, fireOutAt, resolvedAt`, plus:

- `deployedStaffIds` - ids of staff currently active on the incident.
- `staff` - every staff assignment (active and released): `staffId, name, role, assignedAt, releasedAt`.
- `items` - every item assignment: `itemId, name, category, quantity, assignedAt, releasedAt, returnCondition`.
- `alarmHistory` - `alarmLevel, raisedAt` in order.

Staff - /api/staff

Method	Path	Purpose
GET	`/?availability=AVAILABLE`	List staff; availability filter is optional.
GET	`/{id}`	One staff member.
POST	`/`	Create (201). Body: name*, role*, contactNumber, address, employed, trainings[], certifications[]. New staff start UNAVAILABLE.
PATCH	`/{id}`	Partial update; also accepts availability.
DELETE	`/{id}`	Delete (204). Blocked (409) if the member has any assignment history or a linked account; set employed to false instead.

Staff rules: DISPATCHED cannot be set by hand; availability cannot change while the member is assigned to an incident; a DISPATCHED member cannot be marked not employed. Response fields: `id, name, role, contactNumber, address, employed, availability, trainings, certifications, dateJoined`.

Items - /api/items

Method	Path	Purpose
GET	`/?category=ENGINE&availableOnly=true`	List; both filters optional. availableOnly keeps GOOD items that have at least one free unit.
GET	`/{id}`	One item.
GET	`/search?name=`	Case-insensitive name match (404 if none).
POST	`/`	Create (201). Body: name*, description, quantity (>= 0), condition*, category (default OTHER), serialNumber.
PATCH	`/{id}`	Partial update of name, description, quantity, condition, lastMaintained, category, serialNumber.
POST	`/{id}/increase?amount=N`	Add N to quantity (rejected if the result is below zero or below the dispatched count).
DELETE	`/{id}`	Delete (204). Blocked (409) if the item has assignment history; set condition to RETIRED instead.

Response fields: `id, name, description, quantity, condition, lastMaintained, category, serialNumber, quantityDispatched, quantityAvailable, availability`.

Auth and accounts

Method	Path	Purpose
GET	`/api/auth/registered`	{"registered": true|false}. False means no enabled admin exists yet, so the UI should offer account creation.
POST	`/api/auth/signin`	Body: email*, password*. Returns the account (no token).
POST	`/api/auth/signup`	Body: email*, password* (min 8), displayName, role (default ADMIN). 201.
GET	`/api/accounts, /{id}`	List or fetch accounts (never includes the password hash).
PATCH	`/api/accounts/{id}/enabled?value=`	Enable or disable an account.
PUT	`/api/accounts/{id}/staff/{staffId}`	Link an account to a staff member.
DELETE	`/api/accounts/{id}/staff`	Unlink the staff member.
DELETE	`/api/accounts/{id}`	Delete an account (204).

> **Security note.** There is no token or session authentication yet. Sign-in only checks credentials, every endpoint is open, roles are stored but not enforced, and sign-up lets the caller choose any role. Do not expose this server publicly until authentication is added.

5. Example workflow: a house fire

This follows one incident from setup to resolution. It was run against a local copy of the server, so the responses below are real (trimmed). Assume the server is at `localhost:8080` and each command sends `-H 'Content-Type: application/json'`.

Step 0 - First run: create the admin

```
curl localhost:8080/api/auth/registered                    -> {"registered":false}
curl -X POST localhost:8080/api/auth/signup -d '{"email":"admin@bumberoos.com",
     "password":"changeme123","displayName":"Admin"}'      -> 201, role ADMIN
```

Step 1 - Set up the roster and equipment

Add volunteers and mark the ones on duty AVAILABLE (new staff start UNAVAILABLE), then add equipment. The engine is its own row with quantity 1; the fire suits are one row with quantity 10.

```
curl -X POST /api/staff -d '{"name":"Ana","role":"volunteer","employed":true}'     (repeat for Ben, Cara)
curl -X PATCH /api/staff/1 -d '{"availability":"AVAILABLE"}'                      (repeat for 2 and 3)

curl -X POST /api/items -d '{"name":"Engine 1","quantity":1,"condition":"GOOD",
                            "category":"ENGINE","serialNumber":"ABC 123"}'
curl -X POST /api/items -d '{"name":"Fire Suit","quantity":10,"condition":"GOOD",
                            "category":"FIRE_SUIT"}'
```

Step 2 - Report the incident and send the first responders

One call creates the incident and dispatches Ana, Ben, the engine and four suits. If any part fails, nothing is created.

```
curl -X POST /api/incidents -d '{"nature":"EMERGENCY","alarmLevel":"FIRST_ALARM",
     "incidentType":"Fire","location":"Zone 3","reportedBy":"J. Cruz",
     "staffIds":[1,2],
     "items":[{"itemId":1,"quantity":1},{"itemId":2,"quantity":4}]}'

201 {
  "id": 1, "nature": "EMERGENCY", "status": "ONGOING", "alarmLevel": "FIRST_ALARM",
  "incidentType": "Fire", "location": "Zone 3", "reportedBy": "J. Cruz",
  "deployedStaffIds": [1, 2],
  "staff": [ {"staffId":1,"name":"Ana","role":null,"releasedAt":null},
             {"staffId":2,"name":"Ben","role":null,"releasedAt":null} ],
  "items": [ {"itemId":1,"name":"Engine 1","category":"ENGINE","quantity":1,"releasedAt":null},
             {"itemId":2,"name":"Fire Suit","category":"FIRE_SUIT","quantity":4,"releasedAt":null} ],
  "alarmHistory": [ {"alarmLevel":"FIRST_ALARM","raisedAt":"2026-10-14T10:10:55"} ]
}
```

Ana and Ben are now DISPATCHED, the engine shows DISPATCHED, and the fire suits show PARTIALLY_DISPATCHED with quantityAvailable 6 - none of it set by hand.

Step 3 - Send more help as it grows

```
curl -X PUT   '/api/incidents/1/staff/3?role=Driver'       -> 200, Cara added with role "Driver"
curl -X PUT   '/api/incidents/1/items/2?quantity=6'        -> 200, suits on scene: 4 -> 6
curl -X PATCH /api/incidents/1 -d '{"alarmLevel":"SECOND_ALARM"}'
     -> alarmHistory now: FIRST_ALARM, then SECOND_ALARM (each with its time)
```

Step 4 - What a conflict looks like

```
curl -X PUT '/api/incidents/1/items/2?quantity=11'
     -> 409 {"error":"only 4 of Fire Suit available, 5 requested"}

curl -X POST /api/incidents -d '{"nature":"EMERGENCY","incidentType":"Fire","location":"Z",
     "reportedBy":"x","staffIds":[1]}'
     -> 409 {"error":"Ana is DISPATCHED"}           (and no new incident was created)
```

Step 5 - Pull someone back

```
curl -X DELETE /api/incidents/1/staff/2        -> Ben's assignment gets a releasedAt; deployedStaffIds
                                                  no longer lists him; he is AVAILABLE again
curl '/api/staff?availability=AVAILABLE'       -> Ben is listed and can be sent elsewhere
```

Step 6 - Resolve the incident

Resolving releases everyone and returns all units. Here the engine came back damaged, so we record that.

```
curl -X POST /api/incidents/1/resolve -d '{"itemReturns":[{"itemId":1,"condition":"DAMAGED"}]}'
     -> "status":"RESOLVED", "resolvedAt":"2026-10-14T10:10:55"
```

- Ana and Cara are AVAILABLE again; the suits' quantityDispatched is back to 0.
- Engine 1 is condition DAMAGED, so its availability is UNAVAILABLE and it no longer appears in `GET /api/items?availableOnly=true` until someone repairs it with `PATCH /api/items/1 {"condition":"GOOD"}`.
- The incident keeps its full history: who was assigned, in what role, when they were released, and every alarm level.

Step 7 - Afterwards (optional)

```
curl '/api/incidents?status=RESOLVED&nature=EMERGENCY'     -> past emergencies
curl -X POST /api/incidents/1/reopen                       -> ONGOING again (nobody is re-dispatched)
curl -X DELETE /api/incidents/1                            -> 204 (anything still dispatched is released)
```

Quick reference: who blocks whom

You try to...	Result
Assign staff who is DISPATCHED, UNAVAILABLE or not employed	409 naming the person and the reason
Assign or change items on a RESOLVED incident	409 - reopen it first
Request more units than are free, or a non-GOOD item	409 naming the item
Set an alarm level on a NON_EMERGENCY incident	400
Change availability of someone who is assigned	409 - release them first
Delete staff or an item that has ever been assigned	409 - use employed=false / condition RETIRED
Cut an item's quantity below what is dispatched	409